

✓ SCS 3546: Deep Learning

Assignment 3: Contextualized Word Embeddings

Assignment Description

Search Engines are a standard tool for finding relevant content. The calculation of similarity between textual information is an important factor for better search results.

Objectives

Your goal in this assignment is to calculate the textual similarity between queries and the provided sample documents, using a variety of NLP approaches.

In achieving the above goal, you will also:

- Demonstrate how to preprocess text and embed textual data.
- Compare the results of textual similarity scoring between traditional and deep-learning based NLP methods.

Techniques to Demonstrate

The techniques you will use to compute the similarity scores are:

- 1. TF-IDF.
- 2. Semantic similarity using GloVe word vectors.
- 3. Semantic similarity using a BERT-based model.

Feel Free to Choose Your Own Approach

How you go about demonstrating each of the above techniques is up to you. You are not expected to use any particular library. The code below is just meant to provide you with some guidance to get started. You **do**, however, need to demonstrate obtaining similarity scores **with all 3 techniques above**, but how you go about doing this is totally up to you. The evaluation will be based on your ability obtain results using all three techniques, plus your discussion/comparison of any differences you observe.

✓ Setup and Data Import

```
import json
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

import re
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize

import nltk
nltk.download('punkt')
nltk.download('wordnet')
nltk.download('omw-1.4') # Lemmatizer word database
```

Show hidden output

```
# Install gensim
!pip install gensim
```

Show hidden output

```
import gensim
# print(gensim.__version__)
from gensim.utils import simple_preprocess
```

```
# Mount Google Drive
from google.colab import drive
```

```
drive.mount('/content/drive')
```

Mounted at /content/drive

```
# Copy json-file from Google Drive to current working directory
!cp "/content/drive/MyDrive/Colab Notebooks/SCS_3546_Deep Learning/Assignment 3 - Contextualized Word Embeddings/sample_repo_repository.json" .
```

▼ Convert to Pandas DataFrame

```
# Load the JSON
with open("sample_repository.json", "r") as f:
    repo_data = json.load(f)

# Convert to DataFrame
df = pd.DataFrame(repo_data["data"], columns=["Title", "Content"])

# Extract documents for vectorization
documents = df["Content"].tolist()
```

▼ Inspect Data

```
# Show the first few entries
df.head(33)
```

	Title	Content
0	Pomegranate Bhagwa	Fresh Pomegranate from Anushka Avni Internatio...
1	Pomegranate Arakta	Fresh Pomegranate Arakta from Anushka Avni Int...
2	About Us	About Us Anushka Avni International (AAI) take...
3	Contact Us	About Us Anushka Avni International (AAI) take...
4	White Onions	White Onions from Anushka Avni International F...
5	Video Gallery	Anushka Avni International (AAI) takes pleasur...
6	Food classes	To a botanist, a fruit is an entity that devel...
7	Nutrition	Nutrition is the biochemical and physiological...
8	Nutrients	Nutrients are substances used by an organism t...
9	Diet	In nutrition, the diet of an organism is the s...
10	Canada's Food Guide	Canada's Food Guide is a nutrition guide produ...
11	Major Market	UK, Nether Land, Russia, Canada, HongKong, Mal...
12	Canadian Industry Statistics	Canadian Industry Statistics (CIS) analyses in...
13	Red Onions	Red Onions from Anushka Avni International Red...
14	Grapes Sonaka Seedless	Berry Size: 18mm and above Packaging Packing S...
15	Pink Onions	Pink Onions from Anushka Avni International Pi...
16	About Us	Anushka Avni International (AAI) takes pleasur...
17	Small Onions	We are one of the leading organizations engage...
18	Tomatoes	Fresh Tomatoes from Anushka Avni International...
19	Downloads	Anushka Avni International (AAI) takes pleasur...
20	Welcome to Anushka Avni International (AAI)	Welcome to Anushka Avni International (AAI) An...
21	Grapes Thompson Seedless	Thomson Seedless These grapes are seedless, sw...
22	Grapes Black Sharad Seedless	Black Sharad Seedless Grapes Crisp to taste, t...
23	Grapes Flame / Red Seedless	Flame / Red seedless grapesFlame grapes are on...
24	Python dict	A Dictionary in Python is an unordered collect...
25	history of botany	Botany originated in prehistory as herbalism w...
26	Semantic similarity	Semantic similarity is a metric defined over a...
27	botany	Botany, also called plant science(s), plant bi...
28	Ford Bronco	The Ford Bronco is a model line of sport utili...
29	List of fruit dishes	Fruit dishes are those that use fruit as a pri...
30	Neuro linguistic programming	Neuro linguistic programming (NLP) is a pseudo...

```
# Check the number of rows and columns
df.shape
```

(32, 2)

```
# Get basic statistics (non-numeric here, but still useful)
df.describe(include='all')
```

	Title	Content
count	32	32
unique	31	30
top	About Us	About Us Anushka Avni International (AAI) take...
freq	2	2

```
# Check for any missing content
df.isnull().sum()
```

```
0
Title    0
Content  0

dtype: int64
```

```
# Look for duplicated rows (based on content)
df.duplicated(subset="Content").sum()
```

```
2
```

```
# Optionally, look at the longest documents
df["Content Length"] = df["Content"].apply(len)
df.sort_values(by="Content Length", ascending=False).head()
```

	Title	Content	Content Length
16	About Us	Anushka Avni International (AAI) takes pleasur...	827
1	Pomegranate Arakta	Fresh Pomegranate Arakta from Anushka Avni Int...	755
25	history of botany	Botany originated in prehistory as herbalism w...	754
10	Canada's Food Guide	Canada's Food Guide is a nutrition guide produ...	710
0	Pomegranate Bhagwa	Fresh Pomegranate from Anushka Avni Internatio...	531

```
#View duplicate rows
df[df.duplicated(subset="Content", keep=False)]
```

	Title	Content	Content Length
2	About Us	About Us Anushka Avni International (AAI) take...	353
3	Contact Us	About Us Anushka Avni International (AAI) take...	353
5	Video Gallery	Anushka Avni International (AAI) takes pleasur...	286
19	Downloads	Anushka Avni International (AAI) takes pleasur...	286

Experiment 1: TF-IDF

Term **F**requency - Inverse **D**ocument **F**requency (TF-IDF) is a traditional NLP technique to look at words that appear in both pieces of text, and score them based on how often they appear. For this experiment, you are free to use the TF-IDF implementation provided by scikit-learn.

TF-IDF without preprocessing

```
# Initialize and fit TF-IDF vectorizer
tfidf_vectorizer = TfidfVectorizer(stop_words='english')
tfidf_matrix = tfidf_vectorizer.fit_transform(documents)
```

```
print(f"TF-IDF matrix shape: {tfidf_matrix.shape}")
```

```
TF-IDF matrix shape: (32, 596)
```

```
#TF-IDF matrix shape: (n_documents, n_unique_terms)
#32 = number of documents
#596 = number of unique terms across all documents after stop-word filtering
```

```
# Define function to calculate similarity score and rank the results
def get_top_matches_tfidf_df(query, tfidf_vectorizer, tfidf_matrix, df, top_k=5):
    """
    Compute TF-IDF cosine similarity between a query and documents.
    Returns results as a pandas DataFrame.
    """
    query_vec = tfidf_vectorizer.transform([query])
    similarity_scores = cosine_similarity(query_vec, tfidf_matrix).flatten()
    top_indices = similarity_scores.argsort()[-top_k:][::-1]
```

```
result_df = pd.DataFrame({
    "Rank": range(1, top_k + 1),
    "Title": [df.iloc[idx]["Title"] for idx in top_indices],
    "Similarity": [similarity_scores[idx] for idx in top_indices],
    "Snippet": [df.iloc[idx]["Content"][:200] for idx in top_indices]
})

return result_df
```

```
def compute_word_frequency(query, df, text_column="Content", top_k=5):
    query_words = query.lower().split()
    freq_data = []

    for i, row in df.iterrows():
        content_words = row[text_column].lower().split()
        word_counts = {word: content_words.count(word) for word in query_words}
        total = sum(word_counts.values())
        freq_data.append({"Title": row["Title"], "Total": total, **word_counts})

    result_df = pd.DataFrame(freq_data)
    return result_df.sort_values(by="Total", ascending=False).head(top_k)
```

Query 1 - "fruits"

Similarity (TF-IDF)

```
# Define and run Query 1
query_1 = "fruits"
results_tfidf_q1 = get_top_matches_tfidf_df(query_1, tfidf_vectorizer, tfidf_matrix, df)
print("Top results for Query 1: 'fruits'")
display(results_tfidf_q1)
```

Top results for Query 1: 'fruits'				
	Rank	Title	Similarity	Snippet
0	1	Food classes	0.179036	To a botanist, a fruit is an entity that devel...
1	2	Canada's Food Guide	0.081038	Canada's Food Guide is a nutrition guide produ...
2	3	fruit serving bowl	0.000000	A fruit serving bowl is a round dish or contai...
3	4	Neuro linguistic programming	0.000000	Neuro linguistic programming (NLP) is a pseudo...
4	5	Pomegranate Arakta	0.000000	Fresh Pomegranate Arakta from Anushka Avni Int...

Word Frequency (Raw)

```
# Query 1: "fruits"
query_1 = "fruits"

# Compute word frequency using the reusable function
freq_results_q1 = compute_word_frequency(query_1, df, top_k=5)

# Display top documents
print(f"Top documents by raw word frequency for query: '{query_1}'")
display(freq_results_q1)
```

Top documents by raw word frequency for query: 'fruits'			
	Title	Total	fruits
0	Pomegranate Bhagwa	0	0
1	Pomegranate Arakta	0	0
30	Neuro linguistic programming	0	0
29	List of fruit dishes	0	0
28	Ford Bronco	0	0

```
# Since no preprocessing is applied yet, word counts may miss common matches - e.g., "fruits" won't match "fruits." or
# This highlights why text cleaning (like punctuation removal) is important.
```

Query 2 - "vegetables"

Similarity (TF-IDF)

```
# Define and run Query 2
query_2 = "vegetables"
results_tfidf_q2 = get_top_matches_tfidf_df(query_2, tfidf_vectorizer, tfidf_matrix, df)
print("Top results for Query 2: 'vegetables'")
display(results_tfidf_q2)
```

Top results for Query 2: 'vegetables'				
	Rank	Title	Similarity	Snippet
0	1	Canada's Food Guide	0.090708	Canada's Food Guide is a nutrition guide produ...
1	2	fruit serving bowl	0.000000	A fruit serving bowl is a round dish or contai...
2	3	Neuro linguistic programming	0.000000	Neuro linguistic programming (NLP) is a pseudo...
3	4	Pomegranate Arakta	0.000000	Fresh Pomegranate Arakta from Anushka Avni Int...
4	5	About Us	0.000000	About Us Anushka Avni International (AAI) take...

Word Frequency (Raw)

```
# Query 2: "vegetables"
query_2 = "vegetables"

# Compute word frequency using the reusable function
freq_results_q2 = compute_word_frequency(query_2, df, top_k=5)

# Display top documents
print(f"Top documents by raw word frequency for query: '{query_2}'")
display(freq_results_q2)
```

Top documents by raw word frequency for query: 'vegetables'			
	Title	Total	vegetables
10	Canada's Food Guide	1	1
0	Pomegranate Bhagwa	0	0
17	Small Onions	0	0
30	Neuro linguistic programming	0	0
29	List of fruit dishes	0	0

```
# The document "Canada's Food Guide" contains the word "vegetables" once.
# All other documents show a frequency of zero.
# This likely undercounts relevant content due to the lack of text preprocessing.
# For example, words like "vegetables." or "vegetables," may have been missed.
```

Query 3 - "healthy foods in Canada"

Similarity (TF-IDF)

```
# Define and run Query 3
query_3 = "healthy foods in Canada"
results_tfidf_q3 = get_top_matches_tfidf_df(query_3, tfidf_vectorizer, tfidf_matrix, df)
print("Top results for Query 3: 'healthy foods in Canada'")
display(results_tfidf_q3)
```

Top results for Query 3: 'healthy foods in Canada'

	Rank	Title	Similarity	Snippet
0	1	Canada's Food Guide	0.639988	Canada's Food Guide is a nutrition guide produ...
1	2	Diet	0.313719	In nutrition, the diet of an organism is the s...
2	3	Canadian Industry Statistics	0.111874	Canadian Industry Statistics (CIS) analyses in...
3	4	Major Market	0.087720	UK, Nether Land, Russia, Canada, HongKong, Mal...
4	5	Ford Bronco	0.077690	The Ford Bronco is a model line of sport utili...

Word Frequency (Multi-word)

```
# Query 3: "healthy foods in Canada"
query_3 = "healthy foods in Canada"

# Compute per-word frequency for top documents
freq_results_q3 = compute_word_frequency(query_3, df, top_k=5)

# Display results
print(f"Top documents by raw word frequency for query: '{query_3}'")
display(freq_results_q3)
```

Top documents by raw word frequency for query: 'healthy foods in Canada'

	Title	Total	healthy	foods	in	canada
10	Canada's Food Guide	12	4	2	3	3
16	About Us	4	0	0	4	0
28	Ford Bronco	4	0	0	4	0
22	Grapes Black Sharad Seedless	3	0	0	3	0
25	history of botany	3	0	0	3	0

```
# The query was split into four individual words: "healthy", "foods", "in", and "canada".
# The document “Canada's Food Guide” contains all four words, with a total frequency of 12.
# Other documents contain only partial matches
# Raw frequency results are affected by the lack of text preprocessing (e.g., ` "Canada." ` or ` "foods," ` would be missec
```

TF-IDF with preprocessing

```
# Preprocesses raw text into clean, lemmatized tokens by removing noise, stopwords, and punctuation.
# Unified across all experiments to ensure consistent input for TF-IDF and GloVe

# Ensure stopwords are downloaded
import nltk
nltk.download('stopwords')
nltk.download('wordnet')

# Initialize tools
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess(text):
    """
    Clean and tokenize input text:
    - Lowercase
    - Remove punctuation
    - Remove stopwords
    - Lemmatize
    - Return list of tokens
    """
    # Remove HTML-like tags (e.g., <img>)
    text = re.sub(r'<[^>]+>', ' ', text)

    # Remove punctuation
    text = re.sub(r"[^\\w\\s]", " ", text)

    # Lowercase
    text = text.lower()
```

```
# Tokenize using gensim's cleaner
tokens = simple_preprocess(text, deacc=True, min_len=2)

# Remove stopwords and lemmatize
clean_tokens = [lemmatizer.lemmatize(tok) for tok in tokens if tok not in stop_words]

return clean_tokens
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
```

```
# Preprocess each document and join tokens into strings for TF-IDF input
# Fit and transform the cleaned text into a TF-IDF feature matrix

tfidf_input = [" ".join(preprocess(doc)) for doc in documents]
vectorizer = TfidfVectorizer() # no need for `stop_words='english'`
tfidf_matrix = vectorizer.fit_transform(tfidf_input)
```

Query 1 - "fruits"

Similarity (TF-IDF)

```
# Since the document text was preprocessed and lemmatized, we also need to lemmatize the query (e.g., use "fruit" instead of "fruits")
# If we don't, the query term may not match the cleaned vocabulary, resulting in zero similarity scores.

# Define and run Query 1 using preprocessed content
query_1_clean = "fruit"

# Run the query against the TF-IDF matrix and vectorizer
results_tfidf_q1_clean = get_top_matches_tfidf_df(query_1_clean, vectorizer, tfidf_matrix, df)

print("Top results for Query 1 (using cleaned text): 'fruits'")
display(results_tfidf_q1_clean)
```

Top results for Query 1 (using cleaned text): 'fruits'				
	Rank	Title	Similarity	Snippet
0	1	List of fruit dishes	0.472240	Fruit dishes are those that use fruit as a pri...
1	2	Food classes	0.266151	To a botanist, a fruit is an entity that devel...
2	3	Pomegranate Bhagwa	0.153205	Fresh Pomegranate from Anushka Avni Internatio...
3	4	fruit serving bowl	0.098235	A fruit serving bowl is a round dish or contain...
4	5	Canada's Food Guide	0.057145	Canada's Food Guide is a nutrition guide produ...

Word Frequency (Raw)

```
# Since the document text was preprocessed and lemmatized, we also need to lemmatize the query (e.g., use "fruit" instead of "fruits")
# If we don't, the query term may not match the cleaned vocabulary, resulting in zero word count.

# Query 1: "fruits"
query_1_clean = "fruit"

# Recompute document-level token lists
df["Processed_Tokens"] = df["Content"].apply(preprocess)

# Convert tokens back to string
df["Processed_Text"] = df["Processed_Tokens"].apply(lambda tokens: " ".join(tokens))

# Compute word frequency using updated text column
freq_results_q1_clean = compute_word_frequency(query_1_clean, df, text_column="Processed_Text", top_k=5)

# Display results
print(f"Top documents by raw word frequency for query (using cleaned text): '{query_1_clean}'")
display(freq_results_q1_clean)
```

Top documents by raw word frequency for query (using cleaned text): 'fruit'

	Title	Total	fruit
29	List of fruit dishes	3	3
0	Pomegranate Bhagwa	2	2
6	Food classes	2	2
1	Pomegranate Arakta	1	1
10	Canada's Food Guide	1	1

```
# Text preprocessing using punctuation removal and lemmatization led to noticeably improved performance in both:
# TF-IDF similarity scoring
# Raw word frequency counts

# Without preprocessing:
# Relevant words like "fruits" were often missed due to punctuation (e.g., "fruits.") or inflected forms.
# Similarity scores were occasionally non-zero but inconsistent, and word frequency counts returned zero in all docu

# With preprocessing:
# Query and document terms were consistently aligned (e.g., "fruits" -> "fruit").
# TF-IDF matched more relevant documents with higher similarity scores.

# Word frequency counts correctly reflected actual term usage in context.
# This highlights the importance of text standardization (especially lemmatization and punctuation removal) in improvir
```

Query 2 - "vegetables"

Similarity (TF-IDF)

```
# Since the document text was preprocessed and lemmatized, we also need to lemmatize the query (e.g., use "vegetable" i
# If we don't, the query term may not match the cleaned vocabulary, resulting in zero similarity scores.

# Define and run Query 2 using preprocessed content
query_2_clean = "vegetable"

# Get top TF-IDF matches for the cleaned query
results_tfidf_q2_clean = get_top_matches_tfidf_df(query_2_clean, vectorizer, tfidf_matrix, df)

print("Top results for Query 2 (using cleaned text): 'vegetable'")
display(results_tfidf_q2_clean)
```

Top results for Query 2 (using cleaned text): 'vegetable'

	Rank	Title	Similarity	Snippet
0	1	Canada's Food Guide	0.085213	Canada's Food Guide is a nutrition guide produ...
1	2	fruit serving bowl	0.000000	A fruit serving bowl is a round dish or contai...
2	3	Neuro linguistic programming	0.000000	Neuro linguistic programming (NLP) is a pseudo...
3	4	Pomegranate Arakta	0.000000	Fresh Pomegranate Arakta from Anushka Avni Int...
4	5	About Us	0.000000	About Us Anushka Avni International (AAI) take...

Word Frequency (Raw)

```
# Since the document text was preprocessed and lemmatized, we also need to lemmatize the query (e.g., use "vegetable" i
# If we don't, the query term may not match the cleaned vocabulary, resulting in zero word count.

# Define and run Query 2 using preprocessed content
query_2_clean = "vegetable"

# Compute word frequency
freq_results_q2_clean = compute_word_frequency(query_2_clean, df, text_column="Processed_Text", top_k=5)

# Display top documents
print(f"Top documents by raw word frequency for query (using cleaned text): '{query_2_clean}'")
display(freq_results_q2_clean)
```

Top documents by raw word frequency for query (using cleaned text): 'vegetable'

	Title	Total	vegetable
10	Canada's Food Guide	1	1
0	Pomegranate Bhagwa	0	0
17	Small Onions	0	0
30	Neuro linguistic programming	0	0
29	List of fruit dishes	0	0

```
# TF-IDF similarity dropped slightly:
# Raw query `"vegetables"` → **0.0907**
# Cleaned query `"vegetable"` → **0.0857**

# Word frequency count remained unchanged – the document contained the word once in both versions.

# Preprocessing did not affect term presence but slightly reduced TF-IDF impact, likely due to `"vegetable"` being more
# This highlights how cleaning improves consistency but may slightly reduce exact-match influence in similarity scoring
```

Query 3 - "healthy foods in Canada"

Similarity (TF-IDF)

```
# Since the document text was preprocessed and lemmatized, we also need to lemmatize the query (e.g., use "healthy food")

# Define Query 3
query_3_clean = "healthy food in canada"

# Compute similarity using the cleaned TF-IDF vectorizer and matrix
results_tfidf_q3_clean = get_top_matches_tfidf_df(query_3_clean, vectorizer, tfidf_matrix, df)

# Display results
print("Top results for Query 3 (using cleaned text): 'healthy foods in Canada'")
display(results_tfidf_q3_clean)
```

Top results for Query 3 (using cleaned text): 'healthy foods in Canada'				
	Rank	Title	Similarity	Snippet
0	1	Canada's Food Guide	0.657569	Canada's Food Guide is a nutrition guide produ...
1	2	Diet	0.276972	In nutrition, the diet of an organism is the s...
2	3	fruit serving bowl	0.130757	A fruit serving bowl is a round dish or contai...
3	4	Canadian Industry Statistics	0.110227	Canadian Industry Statistics (CIS) analyses in...
4	5	Major Market	0.090213	UK, Nether Land, Russia, Canada, HongKong, Mal...

Word Frequency (Multi-word)

```
# Query 3: "healthy food in Canada"
query_3_clean = "healthy food in canada"

# Compute per-word frequency for top documents
freq_results_q3_clean = compute_word_frequency(query_3_clean, df, text_column="Processed_Text", top_k=5)

# Display results
print(f"Top documents by raw word frequency for query (using cleaned text): '{query_3}'")
display(freq_results_q3_clean)

# -----
# # Compute per-word frequency for top documents
# freq_results_q3_clean = compute_word_frequency(query_3_clean, df, text_column="Preprocessed_Content", top_k=5)

# # Display results
# print(f"Top documents by raw word frequency for query (using cleaned text): '{query_3}'")
# display(freq_results_q3_clean)
```

Top documents by raw word frequency for query (using cleaned text): 'healthy foods in Canada'

	Title	Total	healthy	food	in	canada
10	Canada's Food Guide	16	4	7	0	5
31	fruit serving bowl	2	0	2	0	0
28	Ford Bronco	2	0	0	0	2
9	Diet	2	0	2	0	0
11	Major Market	1	0	0	0	1

```
# TF-IDF similarity was slightly higher with preprocessing (0.657 vs. 0.639)
# Preprocessing helped capture more relevant terms:
# "foods" -> "food"
# Improved alignment across documents
# Word frequency also improved:
# Total count in top document increased from 12 -> 16

# Preprocessing enhanced both similarity and frequency accuracy by normalizing term forms and removing punctuation.
```

Experiment 2: Semantic matching using GloVe embeddings

```
import gensim
print(f"gensim version: {gensim.__version__}")
```

Show hidden output

```
import logging
import json
import logging
from re import sub
from multiprocessing import cpu_count
import numpy as np
import gensim.downloader as api
from gensim.utils import simple_preprocess
from gensim.corpora import Dictionary
from gensim.models import TfidfModel
from gensim.similarities import WordEmbeddingSimilarityIndex
from gensim.similarities import SparseTermSimilarityMatrix
from gensim.similarities import SoftCosineSimilarity
```

```
# Make notebook output cleaner and supress less important logs
import logging

# Initialize logging
logging.basicConfig(format='%(asctime)s : %(levelname)s : %(message)s', level=logging.WARNING)
```

1 - Load Pre-trained GloVe Embeddings

```
# Load GloVe vectors only if not already in memory
if 'glove' not in locals():
    print("Loading GloVe embeddings...")
    glove = api.load("glove-wiki-gigaword-100")

similarity_index = WordEmbeddingSimilarityIndex(glove)
```

```
Loading GloVe embeddings...
[=====] 100.0% 128.1/128.1MB downloaded
```

2 - Preprocess Documents (using unified preprocess() function)

```
# Preprocess Documents (using unified preprocess() function)

df["Processed_Tokens"] = df["Content"].apply(preprocess)
```

3 - Calculate similarity score for Query 1 (GloVe)

```
# Define raw query text
query_1_text_exp2 = "fruits"

# Preprocess the query
query_1_tokens_exp2 = preprocess(query_1_text_exp2)

# Get preprocessed corpus from DataFrame
corpus_exp2 = df["Processed_Tokens"].tolist()

# Build dictionary (including query to ensure coverage)
dictionary_q1_exp2 = Dictionary(corpus_exp2 + [query_1_tokens_exp2])

# Build TF-IDF model
tfidf_q1_exp2 = TfidfModel(dictionary=dictionary_q1_exp2)

# Create term similarity matrix using GloVe
similarity_matrix_q1_exp2 = SparseTermSimilarityMatrix(
    similarity_index,
    dictionary_q1_exp2,
    tfidf_q1_exp2
)

# -----
# Convert corpus to Bag-of-Words
corpus_bow_q1_exp2 = [dictionary_q1_exp2.doc2bow(doc) for doc in corpus_exp2]

# Convert query to BoW and apply TF-IDF
query_bow_q1_exp2 = dictionary_q1_exp2.doc2bow(query_1_tokens_exp2)
query_tfidf_q1_exp2 = tfidf_q1_exp2[query_bow_q1_exp2]

# -----
# Compute soft cosine similarity between query and documents
soft_index_q1_exp2 = SoftCosineSimilarity(
    tfidf_q1_exp2[corpus_bow_q1_exp2],
    similarity_matrix_q1_exp2
)

similarities_q1_exp2 = soft_index_q1_exp2[query_tfidf_q1_exp2]

# -----
# Rank top 5 documents by similarity

top_n = 5
top_indices_q1_exp2 = np.argsort(similarities_q1_exp2[::-1][:top_n])
top_docs_q1_exp2 = df.iloc[top_indices_q1_exp2].copy()
top_docs_q1_exp2["Similarity_Exp2_Q1"] = similarities_q1_exp2[top_indices_q1_exp2]

# Include document titles for apples-to-apples comparison
top_docs_q1_exp2["Title"] = df["Title"].iloc[top_indices_q1_exp2].values

# Add rank column for cleaner output
top_docs_q1_exp2["Rank"] = range(1, len(top_docs_q1_exp2) + 1)

# Display results
print("🔍 Top 5 documents for Query 1 ('fruits') - Experiment 2 (Semantic Matching with GloVe):")
display(top_docs_q1_exp2[["Rank", "Title", "Similarity_Exp2_Q1"]])
```

100%|██████████| 531/531 [00:13<00:00, 40.24it/s]

/usr/local/lib/python3.11/dist-packages/gensim/similarities/termsim.py:382: RuntimeWarning: divide by zero encountered in divide

normalized_corpus = np.multiply(corpus, 1.0 / corpus_norm)

/usr/local/lib/python3.11/dist-packages/gensim/similarities/termsim.py:382: RuntimeWarning: invalid value encountered in multiply

normalized_corpus = np.multiply(corpus, 1.0 / corpus_norm)

🔍 Top 5 documents for Query 1 ('fruits') - Experiment 2 (Semantic Matching with GloVe):

	Rank	Title	Similarity_Exp2_Q1
0	1	Pomegranate Bhagwa	0.783179
6	2	Food classes	0.718877
29	3	List of fruit dishes	0.688865
1	4	Pomegranate Arakta	0.655441
22	5	Grapes Black Sharad Seedless	0.649750

```
# Key observations
# GloVe captures deeper meaning:
# "Pomegranate Bhagwa" ranked #3 in TF-IDF but became the top result in GloVe, likely because GloVe recognized it as a

# Shared strong result: "Food classes"
# This document ranked #2 in both methods, indicating that it's both lexically and semantically close to the query "fru

# TF-IDF favors exact word match:
# "List of fruit dishes" is ranked #1 in TF-IDF but drops to #3 in GloVe. This makes sense because it literally contain

# GloVe surfaces related but less obvious content:
# "Grapes Black Sharad Seedless" and "Pomegranate Arakta" appear in the top 5 only under GloVe. These are specific fruit

# GloVe provides higher similarity scores overall:
# GloVe's top scores (e.g., 0.78, 0.71, 0.68) are significantly higher than TF-IDF's (e.g., 0.47, 0.26), reflecting the
```

4 - Calculate similarity score for Query 2 (GloVe)

```
# Define raw query text
query_2_text_exp2 = "vegetables"

# Preprocess the query
query_2_tokens_exp2 = preprocess(query_2_text_exp2)

# Get preprocessed corpus from DataFrame
corpus_exp2 = df["Processed_Tokens"].tolist()

# Build dictionary (including query to ensure coverage)
dictionary_q2_exp2 = Dictionary(corpus_exp2 + [query_2_tokens_exp2])

# Build TF-IDF model
tfidf_q2_exp2 = TfidfModel(dictionary=dictionary_q2_exp2)

# Create term similarity matrix using GloVe
similarity_matrix_q2_exp2 = SparseTermSimilarityMatrix(
    similarity_index,
    dictionary_q2_exp2,
    tfidf_q2_exp2
)

# -----
# Convert corpus to Bag-of-Words
corpus_bow_q2_exp2 = [dictionary_q2_exp2.doc2bow(doc) for doc in corpus_exp2]

# Convert query to BoW and apply TF-IDF
query_bow_q2_exp2 = dictionary_q2_exp2.doc2bow(query_2_tokens_exp2)
query_tfidf_q2_exp2 = tfidf_q2_exp2[query_bow_q2_exp2]

# -----
# Compute soft cosine similarity between query and documents
soft_index_q2_exp2 = SoftCosineSimilarity(
    tfidf_q2_exp2[corpus_bow_q2_exp2],
    similarity_matrix_q2_exp2
)

similarities_q2_exp2 = soft_index_q2_exp2[query_tfidf_q2_exp2]

# -----
# Rank top 5 documents by similarity

top_n = 5
top_indices_q2_exp2 = np.argsort(similarities_q2_exp2[::-1][:top_n])
top_docs_q2_exp2 = df.iloc[top_indices_q2_exp2].copy()

top_docs_q2_exp2["Similarity_Exp2_Q2"] = similarities_q2_exp2[top_indices_q2_exp2]

# Document titles for consistent comparison
top_docs_q2_exp2["Title"] = df["Title"].iloc[top_indices_q2_exp2].values

# Add rank for clear output
top_docs_q2_exp2["Rank"] = range(1, len(top_docs_q2_exp2) + 1)

# -----
# Display results with title, rank, and similarity
print("🔍 Top 5 documents for Query 2 ('vegetables') - Experiment 2 (Semantic Matching with GloVe):")
```

```
display(top_docs_q2_exp2[["Rank", "Title", "Similarity_Exp2_Q2"]])
```

100%|██████████| 531/531 [00:13<00:00, 40.26it/s] 🔍 Top 5 documents for Query 2 ('vegetables') – Experiment 2 (Semantic)

1

2

3

4

5

Rank

Title

Similarity_Exp2_Q2

6

1

Food classes

0.638767

10

2

Canada's Food Guide

0.529754

0

3

Pomegranate Bhagwa

0.478079

1

4

Pomegranate Arakta

0.478079

29

5

List of fruit dishes

0.474893

```
# Key observations:

# GloVe improves match relevance by a lot:
# TF-IDF returned mostly zero similarity scores, with only Canada's Food Guide scoring 0.085, while GloVe produced strong results

# Top GloVe result is semantically related but not literal:
# “Food classes” ranked #1 in Experiment 2 with 0.638, but didn’t appear at all in TF-IDF. GloVe identified it as semantically related

# Canada's Food Guide is a shared match – with stronger semantic support:
# Ranked #1 in TF-IDF and #2 in GloVe, indicating both models saw it as relevant – but GloVe gave it a much stronger connection

# GloVe surfaces non-obvious but valid connections:
# Pomegranate Bhagwa, Pomegranate Arakta, and List of fruit dishes showed up only in GloVe results – highlighting its ability to capture semantic relationships

# TF-IDF fails on semantic generalization:
# TF-IDF couldn't find useful matches beyond literal overlaps – leading to 0.0 scores for 4 of 5 results. GloVe, however, successfully
```

5 - Calculate similarity score for Query 3 (GloVe)

```
# -----
# Define raw query text
query_3_text_exp2 = "healthy foods in Canada"

# Preprocess the query
query_3_tokens_exp2 = preprocess(query_3_text_exp2)

# Get preprocessed corpus from DataFrame
corpus_exp2 = df["Processed_Tokens"].tolist()

# Build dictionary (include query tokens to ensure coverage)
dictionary_q3_exp2 = Dictionary(corpus_exp2 + [query_3_tokens_exp2])

# Build TF-IDF model
tfidf_q3_exp2 = TfidfModel(dictionary=dictionary_q3_exp2)

# Create term similarity matrix using GloVe
similarity_matrix_q3_exp2 = SparseTermSimilarityMatrix(
    similarity_index,
    dictionary_q3_exp2,
    tfidf_q3_exp2
)

# -----
# Convert corpus to Bag-of-Words
corpus_bow_q3_exp2 = [dictionary_q3_exp2.doc2bow(doc) for doc in corpus_exp2]

# Step 8: Convert query to BoW and apply TF-IDF
query_bow_q3_exp2 = dictionary_q3_exp2.doc2bow(query_3_tokens_exp2)
query_tfidf_q3_exp2 = tfidf_q3_exp2[query_bow_q3_exp2]

# -----
# Compute soft cosine similarity
soft_index_q3_exp2 = SoftCosineSimilarity(
    tfidf_q3_exp2[corpus_bow_q3_exp2],
    similarity_matrix_q3_exp2
```

```
)

similarities_q3_exp2 = soft_index_q3_exp2[query_tfidf_q3_exp2]

# -----
# Rank top 5 documents by similarity

top_n = 5
top_indices_q3_exp2 = np.argsort(similarities_q3_exp2[::-1][:top_n])
top_docs_q3_exp2 = df.iloc[top_indices_q3_exp2].copy()

top_docs_q3_exp2["Similarity_Exp2_Q3"] = similarities_q3_exp2[top_indices_q3_exp2]

# Add titles and rank
top_docs_q3_exp2["Title"] = df["Title"].iloc[top_indices_q3_exp2].values
top_docs_q3_exp2["Rank"] = range(1, len(top_docs_q3_exp2) + 1)

# -----
# Display results
print("🔍 Top 5 documents for Query 3 ('healthy foods in Canada') - Experiment 2 (Semantic Matching with GloVe):")
display(top_docs_q3_exp2[["Rank", "Title", "Similarity_Exp2_Q3"]])
```

```
100%|██████████| 531/531 [00:13<00:00, 38.72it/s]
/usr/local/lib/python3.11/dist-packages/gensim/similarities/termsim.py:382: RuntimeWarning: divide by zero encountered in divide
    normalized_corpus = np.multiply(corpus, 1.0 / corpus_norm)
/usr/local/lib/python3.11/dist-packages/gensim/similarities/termsim.py:382: RuntimeWarning: invalid value encountered in multiply
    normalized_corpus = np.multiply(corpus, 1.0 / corpus_norm)
🔍 Top 5 documents for Query 3 ('healthy foods in Canada') - Experiment 2 (Semantic Matching with GloVe):
```

	Rank	Title	Similarity_Exp2_Q3
10	1	Canada's Food Guide	0.916237
9	2	Diet	0.581622
16	3	About Us	0.543729
31	4	fruit serving bowl	0.525945
4	5	White Onions	0.462465

```
# Key observations:

# GloVe significantly boosts semantic relevance:
# In Experiment 2, Canada’s Food Guide scored 0.916, much higher than its 0.657 in TF-IDF – reinforcing that GloVe captures deeper semantic relationships

# Consistent top 2 results show strong agreement:
# Both experiments rank Canada's Food Guide and Diet as top matches, indicating that these documents are not only lexically similar but also semantically relevant

# GloVe surfaces deeper matches:
# About Us and White Onions appear only in GloVe results – showing its ability to associate broader ideas like supplier information with food-related queries

# TF-IDF fails to identify semantically related content:
# TF-IDF includes generic or weakly relevant entries like Canadian Industry Statistics and Major Market – which may not directly relate to healthy foods

# Overall similarity scores are much higher with GloVe:
# Top GloVe scores range from 0.91 to 0.46, compared to TF-IDF’s 0.65 to 0.09, clearly showing GloVe’s superior ability to find semantically relevant documents
```

▼

Experiment 3: BERT Model

Use a BERT model obtain sentence embeddings and calculate the similarity between queries and documents.

Hint: see the Module 07 jupyter notebook for examples of how to work with BERT.

▼

1 - Setup: Load BERT Model + Encode Documents

```
# Install sentence-transformers (run this only once in your Colab/Notebook environment)
#!pip install -U sentence-transformers

# Import the model
from sentence_transformers import SentenceTransformer
```

```
# Load a pre-trained sentence embedding model
model_bert = SentenceTransformer('all-MiniLM-L6-v2') # Fast & accurate

# Prepare raw documents (no preprocessing needed)
documents_raw_exp3 = df["Content"].tolist()

# Encode all documents to sentence embeddings
document_embeddings_exp3 = model_bert.encode(documents_raw_exp3, convert_to_numpy=True, show_progress_bar=True)
```

Show hidden output

2 - Calculate similarity score for Query 1 (BERT)

```
# Define and encode the query
query_1_text_exp3 = "fruits"
query_1_embedding_exp3 = model_bert.encode(query_1_text_exp3, convert_to_numpy=True)

# -----
# Compute cosine similarity between query and all document embeddings
similarities_q1_exp3 = cosine_similarity(
    [query_1_embedding_exp3],
    document_embeddings_exp3
)[0] # Extract from 2D result to 1D array

# -----
# Rank top 5 documents
top_n = 5
top_indices_q1_exp3 = np.argsort(similarities_q1_exp3[::-1][:top_n])
top_docs_q1_exp3 = df.iloc[top_indices_q1_exp3].copy()

top_docs_q1_exp3["Similarity_Exp3_Q1"] = similarities_q1_exp3[top_indices_q1_exp3]
top_docs_q1_exp3["Title"] = df["Title"].iloc[top_indices_q1_exp3].values
top_docs_q1_exp3["Rank"] = range(1, len(top_docs_q1_exp3) + 1)

# -----
# Display results
print("🔍 Top 5 documents for Query 1 ('fruits') - Experiment 3 (BERT Sentence Embeddings):")
display(top_docs_q1_exp3[["Rank", "Title", "Similarity_Exp3_Q1"]])
```

🔍 Top 5 documents for Query 1 ('fruits') - Experiment 3 (BERT Sentence Embeddings):

	Rank	Title	Similarity_Exp3_Q1
6	1	Food classes	0.604093
29	2	List of fruit dishes	0.543635
18	3	Tomatoes	0.457065
31	4	fruit serving bowl	0.436352
23	5	Grapes Flame / Red Seedless	0.404212

```
# Key observations:

# All three models agree on top themes:
# “List of fruit dishes” and “Food classes” appear in the top 3 across all experiments - showing high consensus on documents

# Semantic models outperform TF-IDF in depth:
# GloVe and BERT surface fruit-related products like “Grapes Seedless” and “Pomegranate Arakta”, which TF-IDF misses -

# GloVe delivers the strongest similarity scores:
# Experiment 2 shows the highest top-1 score at 0.78, indicating high confidence in matches. BERT follows (~0.60), while TF-IDF is lowest (~0.40).

# BERT shows nuanced context awareness:
# Experiment 3 brings in “Tomatoes” and “Grapes Flame”, recognizing them as conceptually tied to fruits, even without exact word matches.

# TF-IDF is limited by exact word overlap:
# Experiment 1 ranks “fruit serving bowl” and “Canada’s Food Guide”, likely due to direct term matches - but these are not conceptually related to fruits.
```

3 - Calculate similarity score for Query 2 (BERT)

```
# Define and encode the query
query_2_text_exp3 = "vegetables"
query_2_embedding_exp3 = model_bert.encode(query_2_text_exp3, convert_to_numpy=True)

# -----
# Compute cosine similarity between query and documents
similarities_q2_exp3 = cosine_similarity(
    [query_2_embedding_exp3],
    document_embeddings_exp3
)[0]

# -----
# Rank top 5 documents
top_n = 5
top_indices_q2_exp3 = np.argsort(similarities_q2_exp3[::-1][:top_n])
top_docs_q2_exp3 = df.iloc[top_indices_q2_exp3].copy()

top_docs_q2_exp3["Similarity_Exp3_Q2"] = similarities_q2_exp3[top_indices_q2_exp3]
top_docs_q2_exp3["Title"] = df["Title"].iloc[top_indices_q2_exp3].values
top_docs_q2_exp3["Rank"] = range(1, len(top_docs_q2_exp3) + 1)

# -----
# Display results
print("🔍 Top 5 documents for Query 2 ('vegetables') - Experiment 3 (BERT Sentence Embeddings):")
display(top_docs_q2_exp3[["Rank", "Title", "Similarity_Exp3_Q2"]])
```

🔍 Top 5 documents for Query 2 ('vegetables') - Experiment 3 (BERT Sentence Embeddings):

	Rank	Title	Similarity_Exp3_Q2
6	1	Food classes	0.478733
29	2	List of fruit dishes	0.427988
18	3	Tomatoes	0.391477
17	4	Small Onions	0.382593
8	5	Nutrients	0.354892

```
# Key observations:

# TF-IDF fails due to lack of exact matches:
# In Experiment 1, 4 out of 5 documents received a 0.0 similarity score, showing that TF-IDF could not find documents u

# GloVe significantly improves retrieval:
# Experiment 2 surfaces documents like Food classes, List of fruit dishes, and Pomegranate Arakta - all semantically re

# BERT returns strong, generalizable matches:
# Experiment 3 identifies Tomatoes, Small Onions, and Nutrients as top matches - showing BERT's ability to associate ve

# Consensus on 'Food classes' and 'List of fruit dishes':
# Both GloVe and BERT rank Food classes and List of fruit dishes in the top 2-3, confirming these documents are context

# Semantic models enable deeper understanding:
# While TF-IDF is blind to context, both GloVe and BERT enable conceptual matching. BERT, in particular, finds specific
```

▼ 4 - Calculate similarity score for Query 3 (BERT)

```
# Define and encode the query
query_3_text_exp3 = "healthy foods in Canada"
query_3_embedding_exp3 = model_bert.encode(query_3_text_exp3, convert_to_numpy=True)

# -----
# Compute cosine similarity between query and all documents
similarities_q3_exp3 = cosine_similarity(
    [query_3_embedding_exp3],
    document_embeddings_exp3
)[0]

# -----
# Rank top 5 documents
top_n = 5
top_indices_q3_exp3 = np.argsort(similarities_q3_exp3[::-1][:top_n])
top_docs_q3_exp3 = df.iloc[top_indices_q3_exp3].copy()

top_docs_q3_exp3["Similarity_Exp3_Q3"] = similarities_q3_exp3[top_indices_q3_exp3]
```

```
top_docs_q3_exp3["Title"] = df["Title"].iloc[top_indices_q3_exp3].values
top_docs_q3_exp3["Rank"] = range(1, len(top_docs_q3_exp3) + 1)

# -----
# Display results
print("🔍 Top 5 documents for Query 3 ('healthy foods in Canada') - Experiment 3 (BERT Sentence Embeddings):")
display(top_docs_q3_exp3[["Rank", "Title", "Similarity_Exp3_Q3"]])
```

🔍 Top 5 documents for Query 3 ('healthy foods in Canada') - Experiment 3 (BERT Sentence Embeddings):

	Rank	Title	Similarity_Exp3_Q3
10	1	Canada's Food Guide	0.638773
12	2	Canadian Industry Statistics	0.385496
18	3	Tomatoes	0.348018
29	4	List of fruit dishes	0.301652
8	5	Nutrients	0.246295

Key observations:

All models agree on Canada’s Food Guide as the top result:
It ranks #1 in all experiments, confirming it’s the most directly relevant document. BERT (0.64) and GloVe (0.91) assign the highest similarity scores.

GloVe shows strongest overall relevance signals:
Experiment 2 yields the highest similarity scores, with all top 5 results above 0.46, indicating strong semantic alignment.

BERT highlights nutrition-focused terms with context:
BERT (Exp 3) retrieves Tomatoes, Nutrients, and List of fruit dishes - showing its ability to understand "healthy foods" context.

TF-IDF has shallow understanding beyond surface terms:
Experiment 1 includes weaker results like Major Market or Canadian Industry Statistics, likely due to matching the words "healthy" and "foods".

Semantic models capture richer and broader meaning:
Both GloVe and BERT identify semantically aligned but diverse content, making them better suited for natural language understanding.

Technique Comparison

Compare all three techniques and interpret your findings. Do your best to explain the differences you observe in terms of concepts learned in class (not just the what, but also the how and why one technique produces different results from another).

Compare results for Query 1

```
# Select top 3 results from each experiment
tfidf_top3_q1 = results_tfidf_q1_clean[["Title", "Similarity"]].head(3).reset_index(drop=True)
glove_top3_q1 = top_docs_q1_exp2[["Title", "Similarity_Exp2_Q1"]].head(3).reset_index(drop=True)
bert_top3_q1 = top_docs_q1_exp3[["Title", "Similarity_Exp3_Q1"]].head(3).reset_index(drop=True)

# Build comparison table
query1_comparison = pd.DataFrame({
    "Rank": [1, 2, 3],
    "TF-IDF Title": tfidf_top3_q1["Title"],
    "TF-IDF Score": tfidf_top3_q1["Similarity"],
    "GloVe Title": glove_top3_q1["Title"],
    "GloVe Score": glove_top3_q1["Similarity_Exp2_Q1"],
    "BERT Title": bert_top3_q1["Title"],
    "BERT Score": bert_top3_q1["Similarity_Exp3_Q1"]
})

# Display table
query1_comparison.style.set_properties(**{'text-align': 'left'})
```

	Rank	TF-IDF Title	TF-IDF Score	GloVe Title	GloVe Score	BERT Title	BERT Score
0	1	List of fruit dishes	0.472240	Pomegranate Bhagwa	0.783179	Food classes	0.604093
1	2	Food classes	0.266151	Food classes	0.718877	List of fruit dishes	0.543635
2	3	Pomegranate Bhagwa	0.153205	List of fruit dishes	0.688865	Tomatoes	0.457065

```
# Key observations:

# All models surface conceptually relevant results, but semantic techniques (GloVe, BERT) retrieve results with stronger semantic relevance

# TF-IDF relies on literal token matches - hence “List of fruit dishes” ranks highest, while documents like “Tomatoes” are not retrieved

# GloVe captures word-level semantics - identifying documents like “Pomegranate Bhagwa” and “Food classes” even without literal matches

# BERT goes beyond word presence to capture context - it retrieves “Tomatoes” as semantically relevant even though the word is not present

# Ranking order differs across models due to how they represent meaning:
# TF-IDF: frequency-based, sparse, and shallow
# GloVe: dense vectors with global word co-occurrence patterns
# BERT: contextual embeddings that interpret entire sentence meaning
```

▼ Compare results for Query 2

```
# Select top 3 results from each experiment for Query 2: "vegetables"
tfidf_top3_q2 = results_tfidf_q2_clean[["Title", "Similarity"]].head(3).reset_index(drop=True)
glove_top3_q2 = top_docs_q2_exp2[["Title", "Similarity_Exp2_Q2"]].head(3).reset_index(drop=True)
bert_top3_q2 = top_docs_q2_exp3[["Title", "Similarity_Exp3_Q2"]].head(3).reset_index(drop=True)

# Build comparison table
query2_comparison = pd.DataFrame({
    "Rank": [1, 2, 3],
    "TF-IDF Title": tfidf_top3_q2["Title"],
    "TF-IDF Score": tfidf_top3_q2["Similarity"],
    "GloVe Title": glove_top3_q2["Title"],
    "GloVe Score": glove_top3_q2["Similarity_Exp2_Q2"],
    "BERT Title": bert_top3_q2["Title"],
    "BERT Score": bert_top3_q2["Similarity_Exp3_Q2"]
})

# Display table
query2_comparison.style.set_properties(**{'text-align': 'left'})
```

	Rank	TF-IDF Title	TF-IDF Score	GloVe Title	GloVe Score	BERT Title	BERT Score
0	1	Canada's Food Guide	0.085213	Food classes	0.638767	Food classes	0.478733
1	2	fruit serving bowl	0.000000	Canada's Food Guide	0.529754	List of fruit dishes	0.427988
2	3	Neuro linguistic programming	0.000000	Pomegranate Bhagwa	0.478079	Tomatoes	0.391477

```
# Key observations:

# TF-IDF fails to generalize:
# Only Canada's Food Guide scores above zero, while the rest receive 0.000, showing TF-IDF’s inability to capture relevant semantic information

# GloVe understands semantic proximity:
# GloVe surfaces Food classes and Pomegranate Bhagwa, recognizing them as contextually related to vegetables based on word co-occurrence patterns

# BERT captures contextual meaning:
# BERT highlights Tomatoes and List of fruit dishes, demonstrating its ability to relate the query to actual examples contextually

# GloVe provides the strongest similarity scores:
# With scores above 0.47 for all top 3 results, GloVe delivers more confident semantic matches than BERT (max ~0.48) or TF-IDF (max ~0.085)

# Semantic techniques find more useful results:
# Both GloVe and BERT successfully interpret “vegetables” as a category and retrieve nutritionally or categorically relevant items
```

... Compare results for Query 2